

Project: Flappy Bird with PPO

From Data to Solutions

ETH Zurich

8 August 2026

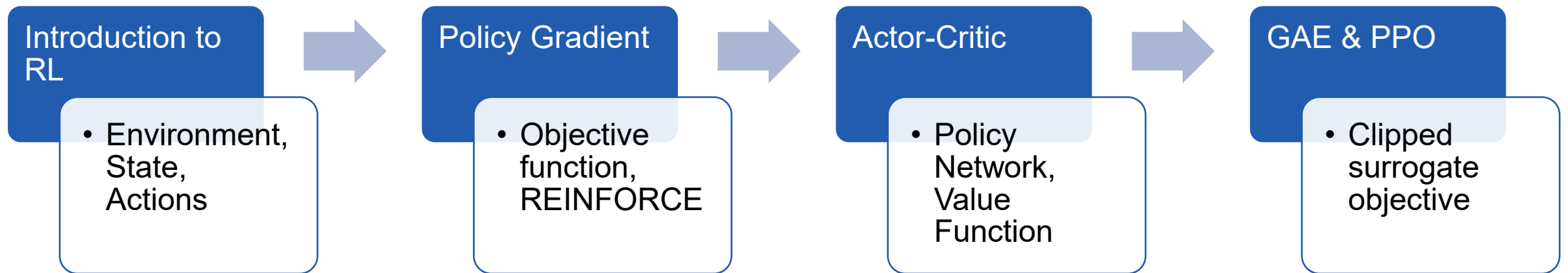
Mattia De Martino, Ankita Ghosh





Recap: What you have
learned

Recap: What you have learned

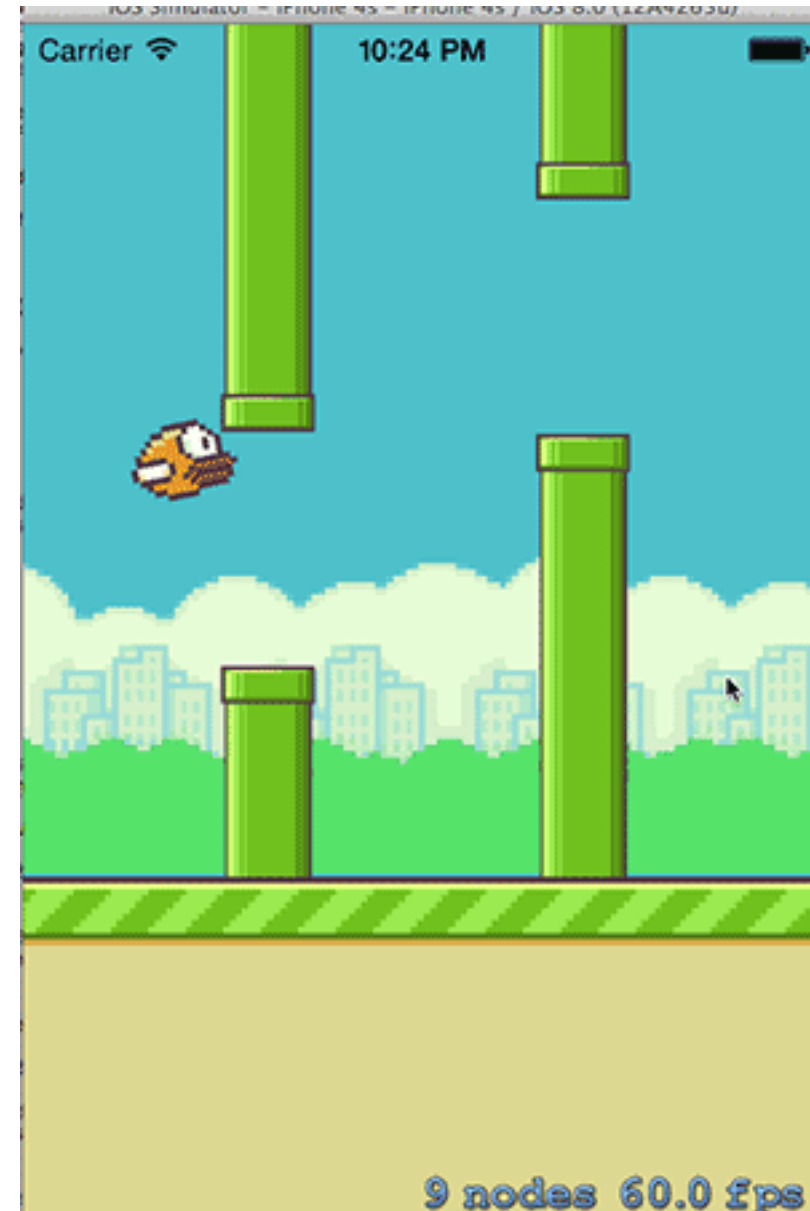


A photograph of a chessboard with a red king piece in the foreground and several black pieces in the background. The background is dark, and the chessboard is lit, creating a dramatic effect. A semi-transparent red rectangle is overlaid on the left side of the image, containing the text 'Project Motivation'.

Project Motivation

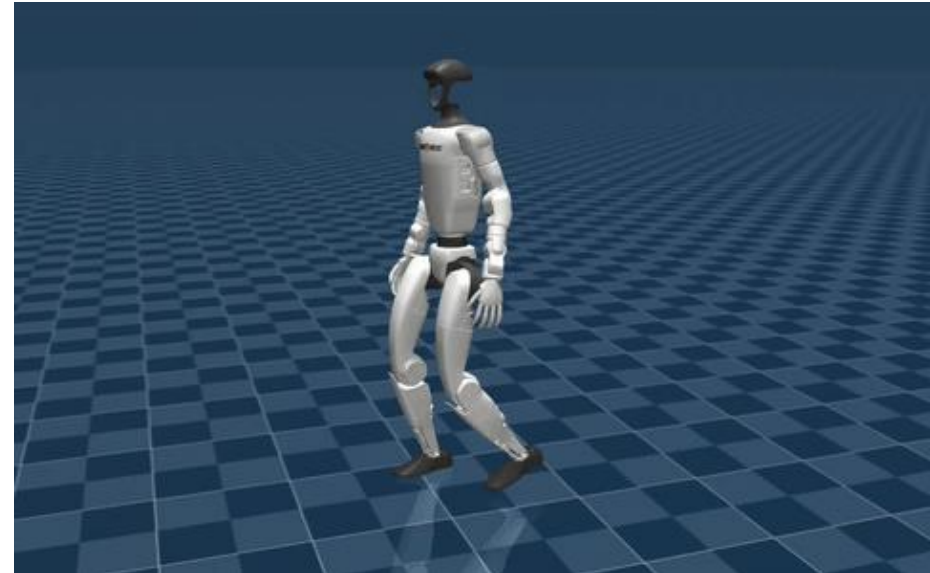
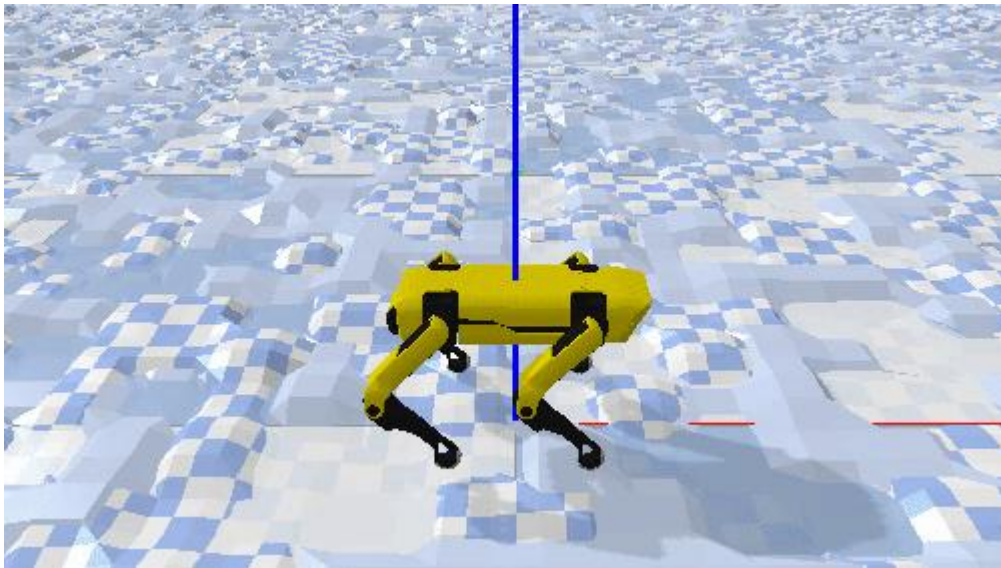
The Game: Flappy Bird

- **Gameplay:** Players must navigate a small yellow bird (Faby) through varying gaps in green.
- **Action:** Players tap the screen to make Faby flap its wings and ascend. If the player stops tapping, gravity pulls the bird down.
- **Objective:** Survival
- **Reward:** Passing through a pair of pipes earns the player one point.
- **Penalty:** A single collision instantly ends the game.



Objective

- Same idea, but this time you *teach* the bird to fly — with PPO — on a clean little discrete-grid version of the game.
- Why?
 - Today, you are training Faby the bird as a single-degree-of-freedom vertical hopper. This serves as a first step to someday train a complex setup, like Spot the robot dog, as a multi-degree-of-freedom quadruped traversing rough and uneven terrain. 😊



Project Setup & Tasks

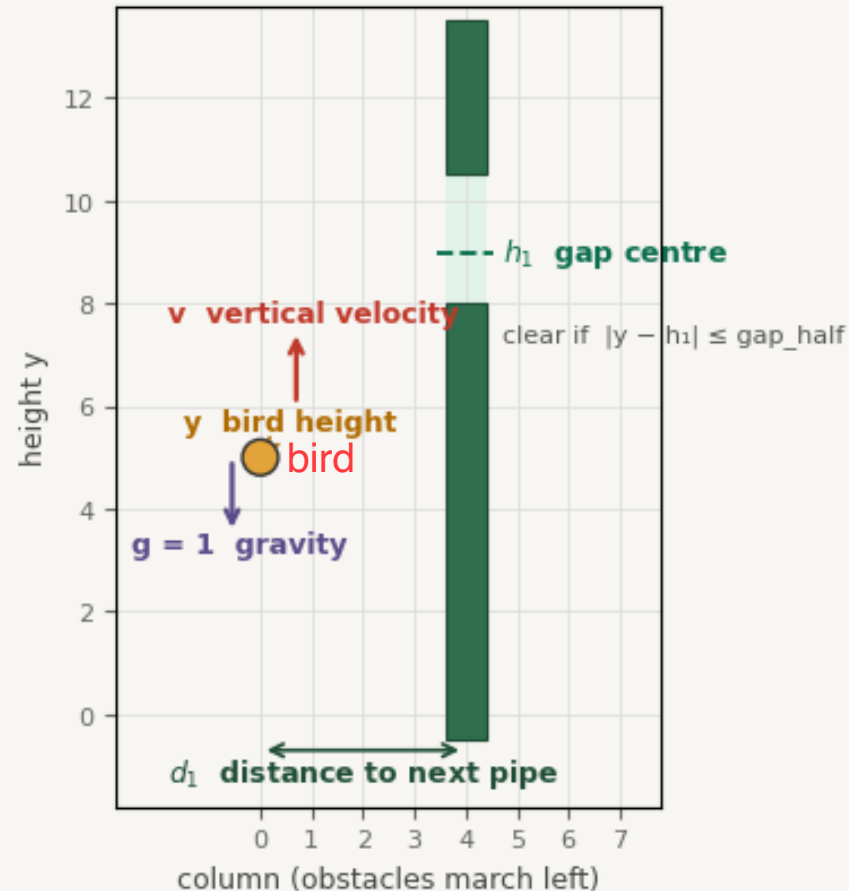


Project Setup

minimize the cost going through the game (policy)

Flappy MDP at a glance — state $(y, v, d_1, h_1, \dots)$ · action $a \in \{0, 1, 2\}$

State at one time step



Actions (discrete impulses)

0 no flap

$u = 0$ cost $\lambda = 0.0$
only gravity $-g (= -1)$

1 weak flap

$u = +2$ cost $\lambda = 0.5$
deterministic kick

2 strong flap

$u = +3 + \epsilon$ cost $\lambda = 0.7$
 $\epsilon \sim \text{Uniform}[-1, +1]$

survived step reward: $r = 1 - \lambda$
dynamics: $y \leftarrow \text{clip}(y + v)$, $v \leftarrow \text{clip}(v + u + \text{noise} - g)$

Project Setup

project goal is that bird learns (using PPO) to fly through the maze

Two Environments

- **Small Instance (Coarse Grid)** small to validate the work
 - State Space: Small enough to fully enumerate.
 - Evaluation: Allows PPO to be compared directly against the true mathematical optimum, rather than a standard baseline.
- **Large Instance (Fine Grid)** large to run and submit
 - State Space: Massive scale (roughly 10^{17} states) sharing the same continuum physics.
 - Evaluation: Requires function approximation (PPO) as the sole tool to learn and evaluate the policy.

Your Tasks



car too heavy mass
drone too much wind
train with more noise

Quick Peak: What is Domain Randomization?

Sim-to-Real Gap

Policies trained in a single, static environment often overfit to its specific quirks, causing them to fail when facing variations or real-world noise.

Method

Randomize physical and environmental parameters (e.g., gravity, pipe gaps, wind, friction) at the start of every training episode.

Outcome

The agent is forced to ignore irrelevant noise, learning a highly generalized, robust policy capable of zero-shot transfer to unseen conditions.

Hand In

- Answer all blocks (code and markdowns) indicated with .
- A training run takes approximately 3-4 minutes. So you have got time for running enough seeds!
- A MP4 file of your trained Faby gets generated at the end so you can visualize your work.
- Submit the completed notebook.

runtime with GPU is better



Enjoy your Project!